Secure data management and storage using decentralised applications (web3)

Aryan Gupta

Dr Mahira Mohamed Mowjoon
Engineering Research Preparation 41030
Faculty of Engineering and Information Technology
University of Technology Sydney
13/11/2023

Introduction 4
Abstract 4
Project Background 4
Project Aim 4
Project Scope 4
Constraints 5
Assumptions 5
Limitations 5
Method 6
Research Conducted 6
Preliminary Research 6
Literature Survey and Review 7
Final Literature Review 8
Development Planning 9
Development Iterations 9
Technical Specifications 9
Frontend 9
NextJs 9
ChakraUI 10
Backend 10
Node Express Server 10
Blockchain 10
Ethereum 10
Database 11
IPFS 11
Project Architecture 11
High Level 11
Smart Contract Interaction 11
Security 12
Certificateless Cryptography 12
Distributed Hash Tables 12
POW Consensus Protocol 13
Findings 14
Development Results 14
User Interface 14
Functionality 14
POW 14
Retrieving Data 14
Adding new Data 15
Editing and Deleting Data 15
Testing Results 15
Smart Contracts 15
Backend Server 16
Discussion 16
Appendices 18
Appendix A: Architecture 18
Appendix A1: High Level Overview 18
Appendix A2: Smart Contract Functionality 18
Appendix A3: Certificateless Cryptography & DHT Integration 19
Appendix A4: POW Consensus Protocol 20
Appendix B: Technology Stack 21
Appendix C: Communication Log 21

Appendix D: User Interface....23
Appendix E: Development Iteration....26
Appendix F: Test Results....27
References.... 28

Introduction

Abstract

Centralised data storage and management has been the industry standard for quite some time. It is a standard which has been entrusted across many domains and industries, holding data of all forms and types from individuals, organisations, governments and more. In recent years, it has become increasingly apparent that centralised data storage and management systems are becoming increasingly vulnerable and prone to compromisation; something in which has been promulgated through a number of high profile cyber attacks and breaches affecting large multinational corporations where sensitive user data has been made public and stolen.

The principles of decentralisation provide an alternative to the ailing centralised data management solution. Through its strengths in scalability, resilience and security, a decentralised solution can eliminate single points of failure within a system ensuring that the system can withstand and remain resilient against malicious attacks and breach. There have been various official literature publications, studies and theoretical applications of decentralisation as a means for data storage, however, there is a lack of practical applications.

The ultimate aim of this project is to provision and provide a robust decentralised data management web application to ensure data integrity and security; whereby risks of cyber attacks, breaches and other malicious activities are minimised and suppressed, ensuring the safety and security of sensitive data and information.

Project Background

Across almost all industries and domains, centralised data storage is prevalent. Centralised data storage has been the norm for quite some time and has been adopted on a large scale basis, whereby sensitive and confidential data en masse is stored. Centralised data storage has served as one of the core foundational means of data management and has been well-established and widely embraced for decades. However, the reliability and integrity of centralised systems have been brought into the spotlight in recent times due to numerous data breaches whereby confidential and sensitive data has been compromised, highlighting the increasing vulnerability of centralised data storage systems. Traditional centralised data storage systems are becoming more and more vulnerable and its impact on society continues to rise.

Project Aim

This research is aimed at finding a solution to protect sensitive data and information from data breaches and cyber attacks. The search for a suitable robust solution has been conducted through a focus on decentralisation as opposed to centralisation, with the proposed solution for this research being a decentralised web application. In tandem with a high degree of security and reliability, the core objective of this research is to provide people with a safe and secure system to store their sensitive data and information and to protect it from unauthorised access and malicious attacks, ensuring complete data security and integrity.

Project Scope

Elements in scope of the project involve the development of a decentralised application inheriting the following principles to ensure safety and security:

- Distributed hash tables
- Certificateless Cryptography
- Peer to peer communication
- POW Consensus Protocol
- Ethereum Blockchain

Constraints

Speed	During high network congestion on the ethereum blockchain there is a possibility of slower speeds.
Browser Compatibility	Not all browsers are compatible with web3 integrations, users may have to manually install these extensions themselves.
Network Limitations	Application performance can be hindered due to the blockchain network connection.
Smart Contract Immutability	Smart contracts cannot be altered after deployment, if changes are required a new contract needs to be deployed.

Assumptions

Metamask / Web3 Extension	Users already have the required browser extensions installed.
Ethereum technical knowledge	Users have basic knowledge of how ethereum works, for example gas and transaction fees.
Internet connectivity	Users have a consistent internet connection which will not drop.

Limitations

Scalability	Scalability concerns and issues are commonly present across all applications, with context to this application implementing the ethereum blockchain the main scalability issue lies within being able to handle large volumes of transactions efficiently.
Data Retrieval Costs	Retrieving data via a blockchain may incur costs, which may introduce some financial strain on users.
Data Modification	Data stored on the blockchain is immutable, ensuring data integrity. Although it can ultimately be changed, it can lead to errors which are not easily addressable.
Third Party Dependencies	This application relies on a third party ethereum wallet, MetaMask to

	facilitate transactions. In the event of MetaMask facing technical issues, it can hinder user experience
Network Forks	In the event of a network fork (when there is a change to the protocol of a blockchain, resulting in a second new blockchain) there may be inconsistencies with user data

Method

Research Conducted

In order to determine the feasibility and viability of this project, additionally providing a solid foundational understanding of this research area. Initially, a preliminary research was conducted in order to develop an understanding on the state of this domain and to grasp an idea of the high level concepts and methods. The preliminary research was then refined and further explored in a literature survey and review ultimately providing the foundation for the final literature review, a culmination of all the research and review conducted, which was included as part of the project proposal.

Preliminary Research

The notion of a decentralised data management system was explored by Ian Clarke in 1999 through his publication to the division of informatics at the University of Edinburgh. Clarke analysed the usage of an algorithm which hypothetically consisted of multiple interconnected nodes each equipped with key-pairs which could be used for data management. Clarke makes reference to his prior work where he constructed an architectural design. Conceptually, Clarke proposes the usage of a binary search tree with nodes containing private keys which hold access to specific elements of data to be used for secure retrieval and storage. Furthermore, Clarke pursues an avenue through which these tree nodes possess the ability to process and handle data requests, aided with a fallback mechanism in order to account for potential delays and timeout events. Ultimately, Clarke pioneers an early hypothesis exploring the potential for a decentralised data management system. Although lacking in security protocols and measures, it can be argued that Clarke was ahead of his time and that there wasn't enough substance in the security domain to support his hypothesis.

(Clarke, 1999)

Under the overarching umbrella of decentralised data storage and management, Davies et al (2006) highlight various potential strategies that can be utilised as a means to provide decentralised data storage through their publication. In contrast to Clarke's paper, Davies et al provide a more in-depth technical analysis. This publication explored the usage of Distributed Hash Tables with context to data replication, allocation capabilities and a hypothetical simulation. Furthermore, the publication included various mathematical and other technical algorithms tasked with creating ideal data creation and allocation functions for use in conjunction with Distributed Hash Tables. Through this technical approach, Davies et al depict an apparent relationship between Distributed Hash Tables and dynamic data replication paving the way for applications around decentralised data storage and management.

(Davies et al. 2006)

Literature Survey and Review

Algarni et al (2020) produced a paper exploring the relationship between blockchain technology and the internet of things (IOT) highlighting the importance and need for decentralised security. The overarching premise of blockchain technology is built upon peer-to-peer communication and secure, transparent interactions between various parties and entities within a decentralised system. This is often achieved through consensus algorithms such as proof-of-work (POW) and a system of distributed ledgers. This publication provides a theoretical approach towards how POW consensus algorithms and blockchain technology can be integrated with the IOT and reveals an in-depth analysis of smart-contract technology. Additionally, Algarni et al also explore other consensus algorithms such as proof-of-proof, paxos and proof-of-authority. Although this publication provides a solid theoretical foundation, it lacks practical applications which make it difficult to understand the role of decentralised security within the context of IOT as this work is purely theoretical.

Algarni et al (2020)

What Algarni et al lacked in their publication was a practical implementation, Li et al (2019) however, were able to provide a practical implementation of how blockchain technology can be integrated with IOT components and systems. Li et al explore various security protocols and methodologies such as certificateless cryptography and distributed hash tables. Certificateless cryptography is explored as a means to provide users with partial keys which can be used to ensure that data is securely stored. In order to store secure addresses on a blockchain, distributed hash tables are proposed which will then interact with the partial private keys generated through certificateless cryptography providing success or failure messages when attempting to interact with the blockchain. Ultimately this publication proposes how certificateless cryptography in tandem with distributed hash tables can be used to secure and protect IOT data, providing an insight into potential applications in the broader software domain.

Li et al (2019)

The InterPlanetary File System (IPFS) is a decentralised data storage mechanism and is analysed and discussed by Bajpai et al (2022) who look into its architectural applications through peer-to-peer networking as a use case for decentralised data storage. By using the IPFS, centralised servers can be avoided and through peer-to-peer networking high volumes of data and requests can be processed with minimal performance issues whilst ensuring a high degree of security and hence the IPFS has been considered as a component for this project. Despite highlighting the exploits of the IPFS and its potential applications in decentralised development, Bajpai et al's publication solely focuses on the IPFS and not the broader implications of decentralised development methodologies.

Bajpai et al (2022)

Domingue and Third (2017) provide an in-depth practical analysis and approach to ensuring data integrity through the use of distributed ledger platforms. The proposed distributed ledger system was theorised to help create a simple decentralised data storage system, utilising cryptographic hashing for data encryption purposes. Due to its popularity and relevance in the domain, Domingue and Third opted to go with the Ethereum blockchain, choosing to integrate their proposed distributed ledger system with ethereum smart contracts. It is clear that this approach was chosen in order to reduce the

risk of single points of failure, in essence moving away from centralisation and moving towards decentralisation

Domingue and Third (2017)

Final Literature Review

After an extensive literature review and analysis of various sources, three main publications stood out as the most relevant for this research project through which some key debates and controversies were analysed.

The first paper in which was by Algarni et al (2020) whose publication aimed to highlight the importance of blockchain technology towards the IOT domain, arguing that the integration of blockchain technology would bring forward many benefits and ultimately improve the security and integrity of user data and devices. This argument was supported by an in-depth analysis into the applications of consensus algorithms, most notably, the proof-of-work (POW) algorithm. In this publication, a POW algorithm is theorised to provide enhanced security to IOT systems by enforcing user participation in validation tasks in return for access to the system. Algarni et al (2020) ultimately provide a blueprint through which IOT systems can adopt in order to have a higher degree of privacy and control over user data through the enforcement of computational work through a POW consensus algorithm. However, POW consensus algorithms do have some shortcomings, the main one being that it is a resource heavy process and opens the door for large entities with large computational power to gain control of a system. Although the POW consensus algorithm has some flaws, Algarni et al (2020) were able to effectively illustrate a method of decentralisation within the context of an IOT system.

Algarni et al (2020)

Like Algarni et al (2020) and their research publication on the potential adoption of blockchain technology with IOT systems, Li et al (2019) also looked into potential integrations of blockchain technology with IOT devices and systems. Their paper focused on the utilisation of certificateless cryptography in tandem with distributed hash tables as the focal security measures to ensure user data is protected and that its data integrity is upheld. This publication was able to successfully exemplify the importance of certificateless cryptography and how it can be used to secure user data through its private and partial key mechanisms. However there is still some debate over the reliability of certificateless cryptography and whether it can be used in large scale applications. This is because in the event of a failure in certificateless cryptography, malicious entities and actors can gain access to and compromise entire systems and components, leaving user data vulnerable.

Li et al (2019)

Finally, Domingue and Third's 2017 publication dives deep into the utility and potential optimisation of distributed ledger systems as they promote its abilities to increase data integrity and security under the umbrella of decentralisation. Their paper dives deep into the importance of data encryption and how cryptographic hashing can be used. Through a combination of data encryption, cryptographic hashing and smart contract technology, a distributed ledger system was concocted ultimately proving decentralisation as a means for data security as feasible. Whilst distributed ledger systems provide various benefits such as enhanced data security, data hashing and encryption, there are still many

apparent complexities involved within its implementation. Additionally, the same goes for applications with smart contract technology.

Domingue and Third (2017)

Development Planning

Development Iterations

In order to ensure consistent development and consistent progress, a development iteration timeline was constructed (refer to Appendix E) which was broken down into 6 main iterations:

1. System Setup and Configuration: This is the initial stage of development where the boilerplate application is setup and all the required dependencies are setup and installed
2. Backend Development: In this iteration the focus is on ensuring that the Ethereum smart contracts are developed and the backend CRUD functions are written.
3. Frontend Development: This iteration focuses on the application of the backend functionality into a user interface, focusing on elements such as visually appealing UI/UX and ease of use
4. Security Measures: This iteration focuses on the security of user data and includes the integration of certificateless cryptography, DHTs and POW consensus protocol
5. Testing and Quality Assurance: By this iteration the application should be fully functional so that in this iteration the focus is on ensuring that the application works as expected and that there are no bugs or issues
6. Deployment and Launch: The final iteration involves the deployment of the application to a hosted server.

Technical Specifications

A high level overview of the technical specifications is divided into 6 sections:

- Frontend
- Backend
- Blockchain
- Database
  Cryptography
- Communication

The technical specifications are visualised in depth in Appendix B.

Frontend

NextJs

NextJs, a react based framework was chosen to develop the frontend framework of the project. NextJs was chosen due to its strengths in performance optimization and rendering. NextJs incorporates a high degree of server-side-rendering (SSR) which improves SEO optimization by rendering content through server-side operations which enables better indexation by search engine crawlers. Additionally through SSR NextJs provides improved performance by increasing client-side workloads by prioritising server-side operations resulting in faster application load speed and

improving overall user experience. Through Incremental Static Regeneration NextJs provides users with instant content following user requests by updating content progressively allowing for high application efficiency. Furthermore, NextJs employs a unified file system which keeps API routes and integration in the same structure as the rest of the application, despite this API logic is isolated and separated providing a clean structure.

(Vercel, 2023)

ChakraUI

In terms of the User Interface component libraries, ChakraUI was chosen due to its consistent and customisable theme. In addition, by providing composable and reusable components, ChakraUI allows for rapid efficient development and a higher degree of maintainability and reusability. Finally, ChakraUI provides responsive design allowing for its UI components to be flexible and cross compatible with all device types.

Backend

Node Express Server

For the application backend, a Node Express server was chosen due to its fast and scalable nature. Node express server's are capable of handling high volumes of connections and support concurrency without the need for multiple threads unlike traditional multi-threaded backend solutions. With integrated middleware support and options extra functionality can be added to the backend application through packages and libraries and also supporting request and response processing operations. Additionally, Node Express Server's support URL mapping is integral in the organisation of API endpoints to become more clear and understandable with the underlying support from various HTTP methods allowing for the creation of industry standard RESTful APIs.

(OpenJS Foundation, n.d.)

Key elements of this project such as Distributed Hash Tables, Certificateless Cryptography and Peer to Peer networking will be implemented via the Node Express backend server aided by its expansive ecosystem of libraries and packages allowing for the seamless integration and development of these critical pieces of the system.

Blockchain

Ethereum

The Ethereum blockchain was chosen due to a wide range of factors. Firstly, in terms of smart contract functionality the ethereum blockchain allows for a high level of automation through the automatic execution of smart contracts upon the achievement of desired pre-conditions, ultimately allowing for cost efficiency by resulting in lower transaction costs and fees. In relation to the project aim, the ethereum blockchain contains a high degree of security by employing core cryptographic fundamentals in order to ensure that transactions remain secure and its data remains immutable. The Ethereum blockchain supports decentralisation through its transparent transactions ensuring that they are incorruptible.

Database

IPFS

For the application database, the *InterPlanetary File System* (IPFS) was chosen. The IPFS is an open source system that supports data management without the need for centralisation, i.e. supporting the aim of this research - decentralisation. The IPFS supports decentralised data storage through a high degree of fault tolerance as it is able to distribute data across multiple nodes, ensuring that the risk of data loss is at a minimal level. Additionally, multiple copies of data are generated in order to ensure availability in the event of node failure. Through historical versioning, the IPFS provides access to older versions of files and data, emulating a form of version control giving users permanent file sharing and persistence.

Furthermore, the IPFS supports data integrity by ensuring that user data remains immutable during transition, by doing this the IPFS adds an extra layer of security to user data ensuring that it cannot be breached or tampered with. As opposed to traditional data retrieval methods based on data location, the IPFS supports content based retrieval allowing for faster data retrieval times further supporting the high degree of data integrity that it upholds. Ultimately, the IPFS is a database in which it is scalable, something which is required for the scope of this research as it supports data management for high volumes of data and is able to do so through efficient data distribution across various nodes and horizontal scaling.

Project Architecture

In order to provide a solid blueprint for the proposed application, various architectural designs were constructed to cover various elements and components of the proposed system.

High Level

A high level architecture for this application was constructed in order to visualise how the different components of this application would interact with each other, refer to Appendix A1. This high level architecture follows the general standard for Decentralised Application Architecture (DApp) as it provides a high level illustration depicting the various integrations between a frontend application, a backend system and a blockchain network. The compatibility of these components amongst each other ultimately determines the security, efficiency and resilience of the application.

Smart Contract Interaction

The smart contract interaction (refer to architecture in Appendix A2) of the application can be broken down into 4 main stages:

1. Initialization:
The smart contract is initialised through a loader function with the role of loading and fetching the users account via a third party application, MetaMask to ensure proper authentication and verification. Once the user account is fetched, the main smart contract is initialised and ready for use. The main smart contract of the application is the one and which handles the secure storage and management of user data

2. User Verification:
The application will then perform verification and authentication checks on the current user to ensure that they are able to interact with the ethereum blockchain. This is done through the browser which will check to see if the application supports ethereum, if it satisfies this requirement, the user is prompted to allow access to it's account, however, if the application does not support ethereum, the user is given a prompt to sign up via a third party application for a suitable account.

3. Loading User Account
The user account is then fetched and a check is carried out to determine whether the user has any data already stored in the smart contract. Since the user has already passed the initialisation and verification, any pre-existing data will be displayed as they have the necessary permissions to view it.

4. Interacting with the smart contract
The user is then able to interact with the smart contract to perform create, read, update and delete (CRUD) operations. These CRUD operations outline the core functionality of the application, however the security protocols and integrations covered in the next section go more into detail on achieving the aim of this research.

Security

Certificateless Cryptography

Certificateless Cryptography is a cryptographic methodology which eliminates the need for certificates when verifying public keys, eliminating any potential escrow problems during the transfer of data. As opposed to traditional public key infrastructure processes and standards, certificateless cryptography provides a unique approach by removing the escrow problem, a problem in which remains quite prevalent. Escrow problems occur when third party private key entities are involved, these entities sometimes handle incoming messages between a user and a system, providing a potential opportunity to access a user's private keys. By eliminating the need for certificates the escrow problem is made redundant as this protocol ensures that no third party entities can gain access to messages being sent between a user and a system, ultimately ensuring that a user's private keys and data remain impenetrable. In the context of this project (refer to Appendix A3), certificateless cryptography is used to encrypt and decrypt data. When encrypting and decrypting data the system will only interact with the user and their private key, there are no certificates or third party services being used, hence no escrow issues. In order to further secure the user data and enforce trust in the decentralisation of the system, distributed hash tables are utilised.

Distributed Hash Tables

Distributed Hash Tables (DHTs) are decentralised distributed systems implementing hash lookup functionality, similar to traditional hash tables. DHTs make use of key value pairs and allow nodes within a system to interact with these key value pairs. DHTs support decentralised data storage as it is able to distribute data across a network through various nodes, reducing single points of failure. Reducing single points of failure ultimately allows DHTs to improve the reliance and availability of the application, ensuring that the users needs are met and that the project's aim is adhered to. Furthemore, DHTs also support scalability and efficiency. DHTs can detect node failures and removals and in these events will provide the system with more nodes and will re-organise the existing network of nodes, scaling with the application. Given the context of this project, the DHTs

interact with the certificateless cryptography encrypt and decrypt functions by providing secure data retrieval methods as highlighted in Appendix A3.

Ultimately, Certificateless Cryptography in tandem with DHTs further enhances the security and decentralisation of the application, ensuring that user data is protected.

POW Consensus Protocol

Proof of Work (POW) is a consensus protocol which has been used since the beginning of the blockchain domain. POW was initially used by Bitcoin where it allowed users (acting as miners) to solve complex computational problems, once these problems were solved they would be allowed to add a block to the blockchain as their reward. By incorporating a problem solving mechanism, POW protocols are able to minimise the risk of malicious activity from users as they are required to commit computational effort and resources in order to interact with the blockchain, ultimately increasing decentralisation and trust within the system. POW ensures that data integrity by enforcing users to commit more work once data has been written, this way stored data remains unchanged and its state preserved over time. Without the barrier of a POW protocol, the system can be overwhelmed by brute force attacks whereby users can overrun the network by uploading high volumes of data, however a POW can act as a rate limiting mechanism by forcing users to commit computational problems each time they wish to interact with the system, preventing any form of malicious activity. Furthermore, the computational power that comes with POW interactions carries a certain degree of cost and mining fees, by doing this the system will remain resilient against Sybil attacks. Sybil attacks occur when a malicious user is able to produce multiple malicious entities through the form of multiple nodes to control a network, however given an implementation of POW it would prove to be very costly as it would spread across each node making it too expensive and less feasible for a malicious actor to penetrate the system to access secure data. Ultimately, the integration of a POW consensus algorithm is arguably a necessary component of this research application as it provides a critical layer of security.

As seen through Appendix A4, the architecture consists of a user/client entity which represents users and clients who interact with and use the system. The User entity proceeds to interact with the user interface where they can send various kinds of requests to handle and manage their data. Once a user has made a successful request through the user interface, the request is then retrieved by the backend server which will handle the encryption functionality and will interface with the smart contract deployed on the ethereum blockchain. The Smart contract contains all the functionality and mechanisms to handle the creation, modification and deletion of user data, each time a piece of smart contract functionality is executed, a POW challenge is triggered to the user and upon a successful POW challenge completion, the data will be stored in the decentralised storage mechanism. The POW mechanism consists of a 5 stage progression (as highlighted in Appendix A4):

1. POW entry: This occurs when a user has attempted to interact with the system and a POW challenge has been triggered
2. Compute Puzzle: This is where the system returns a computational puzzle to the user to be solved
3. Validate Block & Data: If the user is able to solve the presented puzzle, the solution will be validated by the system in order to ensure that the solution is correct and meets the acceptance criteria of the puzzle
4. Add block to chain: If the solution was correct, a new block will be created and added onto the blockchain

5. Reward Miner: The user will be rewarded with a successful interaction with the system

In alignment with the objectives of this project, the proposed architecture provides a blueprint for the end-application with an emphasis on secure decentralisation achieved through smart contracts, certificateless cryptography, distributed hash tables and a Proof of Work consensus protocol.

Findings

Development Results

User Interface

The development of the user interface proved to be a straightforward task with no obstacles in the way. By utilising NextJs and ChakraUI library a concise responsive user interface was constructed. A generic colour scheme was in order to ensure that the application is visually appealing.
Through this user interface, a user's account is loaded and their account identification is displayed on the screen alongside a blank data input component as highlighted in Appendix D - landing page. Once users have added some data, they are then prompted by a web3 provider confirmation. This confirmation provides the users with a summary of the operation they are trying to compute and provides options to either confirm or reject the operation, refer to Appendix D - Web3 Provider Confirmation. Users can similarly edit and delete data and in doing-so they are prompted with confirmation dialogs to ensure that they are performing their desired operation as seen through Appendix D - Edit & Delete data. Moreover, following this confirmation dialogs the users will still be prompted by their web3 provider to confirm their transaction as an extra precautionary measure.

Functionality

POW

As mentioned earlier in the project security section, a POW consensus protocol has been implemented in order to ensure proper user verification and to eliminate any bad entities and actors. The backend server contains two endpoints:

1. Mine: Users will interact with this endpoint before performing any interactions on the application, this is where the system will present a problem to the users which they need to solve
2. SubmitSolution: Users will submit their solution through this endpoint and on successful solutions they will be granted access to the application

Retrieving Data

Through a combination of backend server logic coupled with the User Interface and interactions with the Ethereum network, users are successfully able to retrieve their data. The backend server logic contains encryption and decryption logic to ensure user data is secure and also integrates a certificateless cryptographic mechanism in tandem with a distributed hash table storage class. The encryption and decryption endpoints interface with the User Interface to display and hide data accordingly at a users request. In order to retrieve the data from the Ethereum blockchain network, a javascript function was constructed to load the users Ethereum account and associated data.

Adding new Data

When users add data, the encryption endpoint in the backend server is used to ensure that the user data is properly encrypted. This encryption endpoint makes use of the custom certificateless cryptography and distributed hash table storage mechanisms and classes that have been made. Furthermore, the previously mentioned javascript function is used to ensure that the user data is properly added to the Ethereum blockchain on the correct user account.

Editing and Deleting Data

When editing and/or deleting data, the data is first retrieved from the DHT storage mechanism. The contract operation is then verified with the blockchain and the user is given a prompt on the user interface on whether they want to proceed to edit or delete their data.

Testing Results

Smart Contracts

In order to test the smart contract functionality of the application, a unit test file, written in solidity (language used to construct the smart contract) was used to test the validity and reliability of each function. Testing the smart contract functionality is very important as it ensures that the system is secure and that user data is immutable - upholding the core objectives of this project. Additionally it increases confidence and trust within a system as it ultimately verifies the functionality of a system. Furthermore, test cases are good ways to detect and determine whether there are any vulnerabilities or issues within a system and are a good way to identify if any extra work is required to achieve full functionality.

The test file was set up in such a way so that before each test a new instance of the smart contract is initialised to emulate the live functionality of the application. The test file contains three main test cases:

1. Testing data creation: Verifies that users can successfully create new data and verifies that a user's data count increments properly when they create multiple instances of new data
2. Testing data edit functionality: Verifying that users can successfully edit their data and ensures that the values change correctly
3. Testing data deletion: Verifies that user data when deleted is successfully removed from the smart contract and checks to verify that the data count has properly incremented negatively.

The test cases were run numerous times and proved to be successful - verifying that the project smart contract functionality was correctly implemented, refer to Appendix F for the result set.

Backend Server

Building on from the smart contract test cases, similarly, a test case suite was set up for the backend server using Jest and Supertest. As mentioned before, by testing the backend server of the application it was possible to conduct necessary precautions and tests such as:

- Security and quality assurance: Testing allows us to identify areas in which potential vulnerabilities may lie
- Performance review: Despite successful test results, testing can also highlight potential performance issues through slow test run times

- Functional verification: Testing is another good process to ensure that the provided functionality performs its set tasks

The test suite integrates *beforeAll* and *afterAll* integrations which is a setup and teardown principle where before each test a new instance of the backend server is initialised and closes the server after all tests have been run. The test suite contained four main testing areas:

1. Testing the root backend server endpoint to ensure that the frontend and backend can communicate successfully
2. Testing the encryption functionality to ensure that user data is properly encrypted and that the certificateless cryptography service and DHTs are working as expected
3. Testing the decryption functionality to ensure that users can successfully decrypt their data using a valid key (stemming from the certificateless cryptography service) making sure that they are able to access the correct data
4. POW functionality: Ensuring that the POW consensus protocol integration works as expected and that suitable mining problems are presented to the users and verifying that the system properly checks submitted solutions and provides the correct yes/no response

Similar to the smart contract testing, the backend server test cases were successful and verified that the application was working as expected, refer to Appendix F for the result set.

Discussion

Reflecting back on the outcomes of this research work it has been apparent that this project was indeed a viable solution to the current vulnerabilities of centralised data storage systems and mechanisms. The development of a successful fully functional decentralised web application for the

purpose of data management has argued strongly for the feasibility of a decentralised approach to the broader future of secure data storage.

This project was targeted towards data storage for individuals on a small scale and proved to provide a viable working solution as a minimum viable product. However, in order to tackle the larger scale problem with centralised data storage and management future work in this domain is required . For example, a working solution on a larger scale application among the likes of applications such as banking and financial software where confidential user PII data is stored. Once implemented on a larger scale, confidence in this domain will grow and it will incentivize large organisations and corporations to consider decentralised alternatives for data storage.

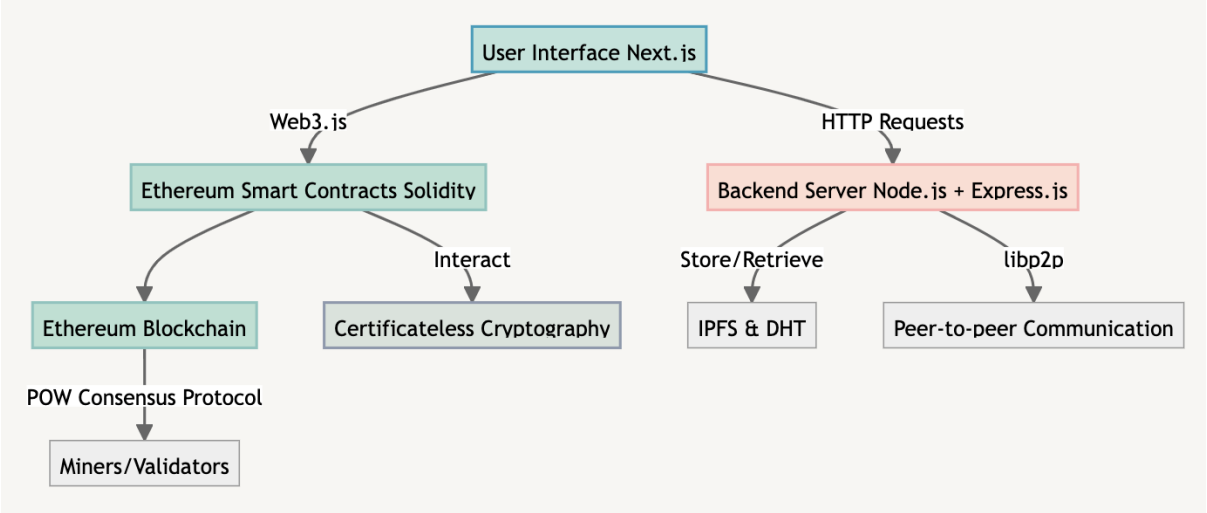
In terms of recommendations for future work into this research area, one main suggestion is to explore integrations with other blockchain networks such as Bitcoin, BNB, Stellar, etc. This is because the Ethereum blockchain can often be quite intensive in terms of resourcing and monetarily and often fluctuates. In order to devise a cost-effective solution, an alternate blockchain network may need to be considered. Additionally, factors such as performance and maintenance also play a part in the need to integrate other blockchain networks.

The purpose of this thesis was to provide a viable solution to the vulnerability of centralised data management. The subsequent research, literature reviews, ideation, development and testing that was documented in this thesis ultimately culminated to form a viable solution. Ultimately, the aim in which to provide and provision a robust decentralised data management web application as a solution to ensure data integrity and security was inevitably achieved.

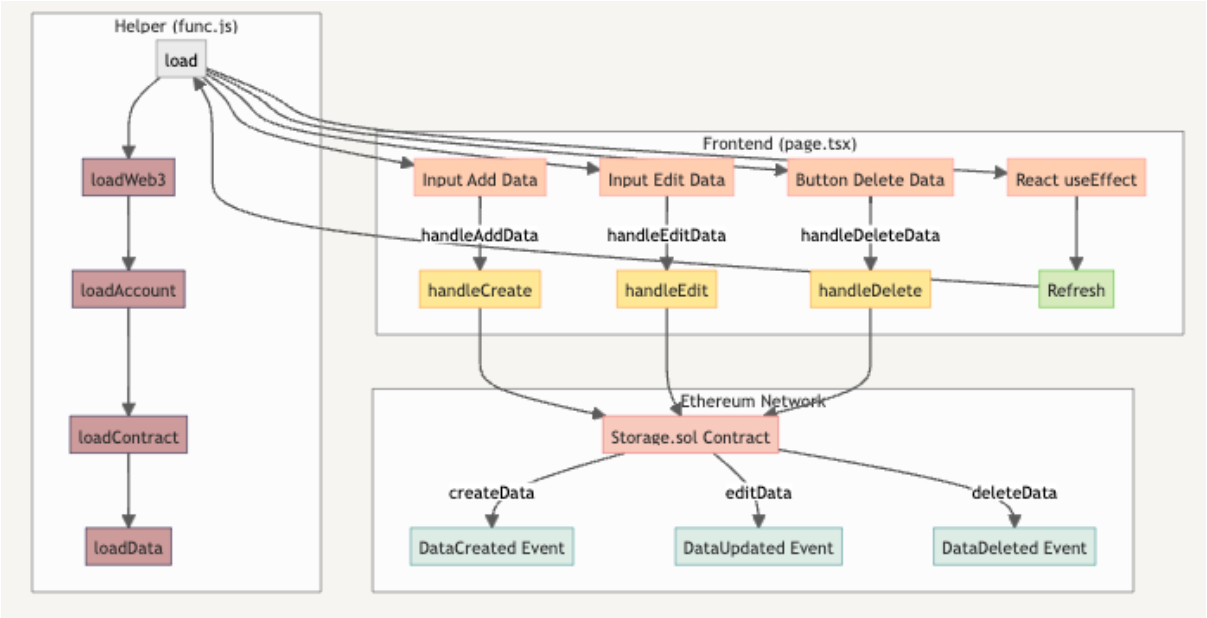
Appendices

Appendix A: Architecture

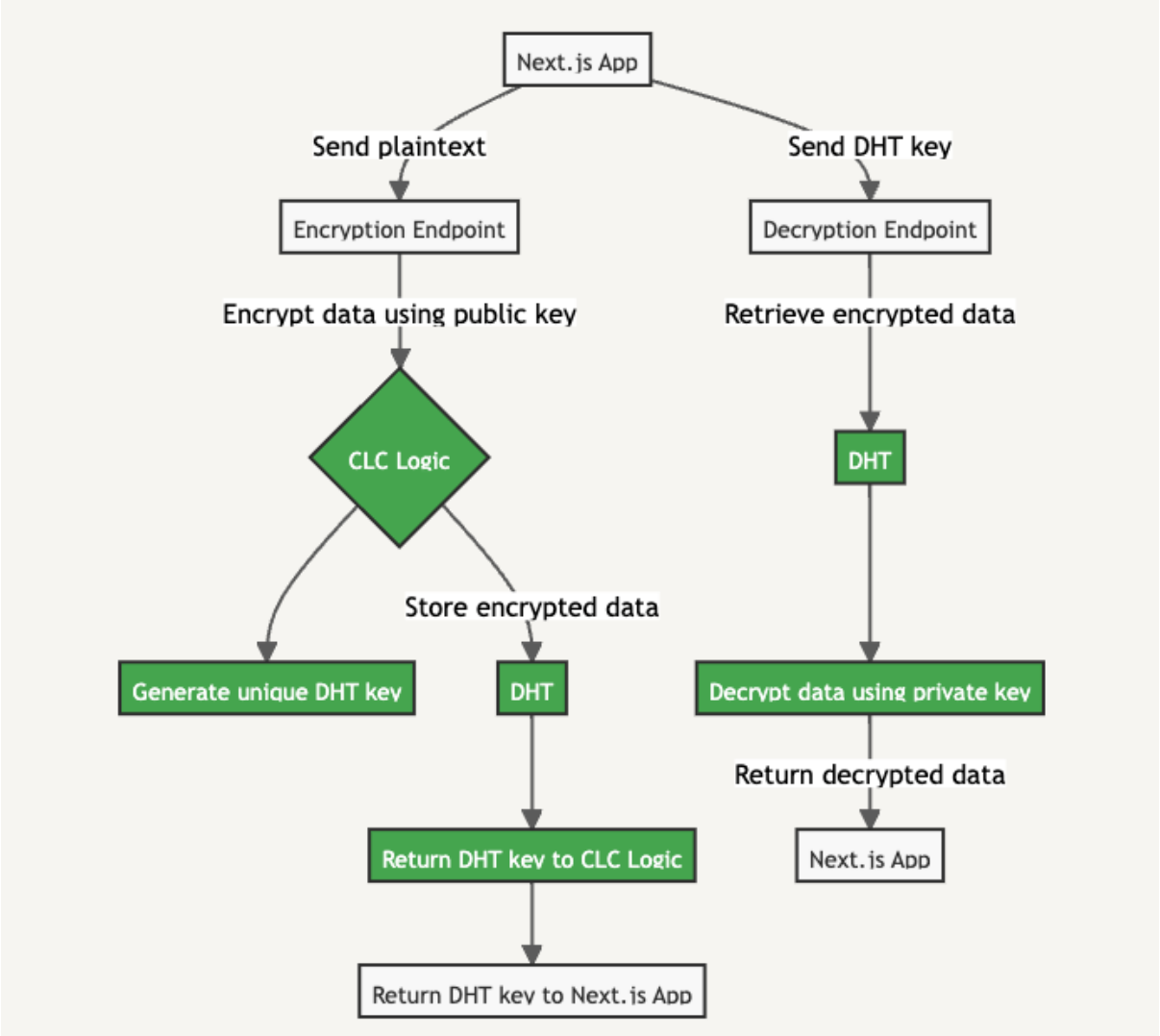
Appendix A1: High Level Overview



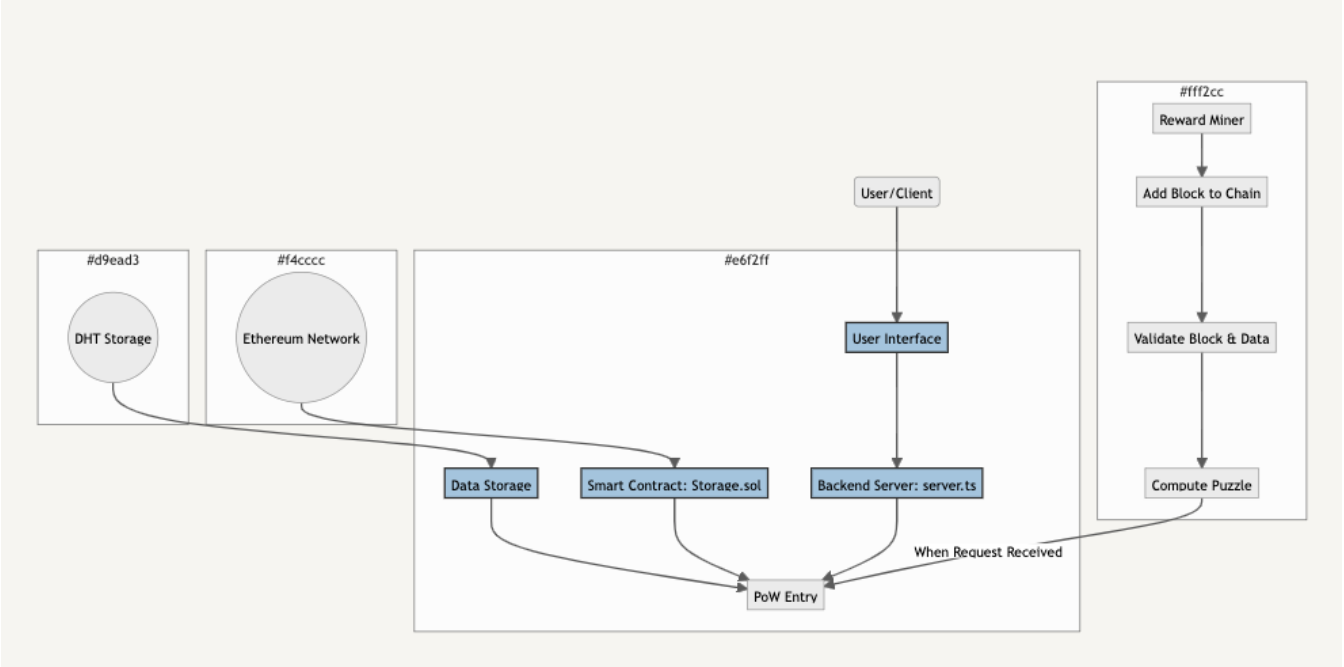
Appendix A2: Smart Contract Functionality



Appendix A3: Certificateless Cryptography & DHT Integration



Appendix A4: POW Consensus Protocol



Appendix B: Technology Stack

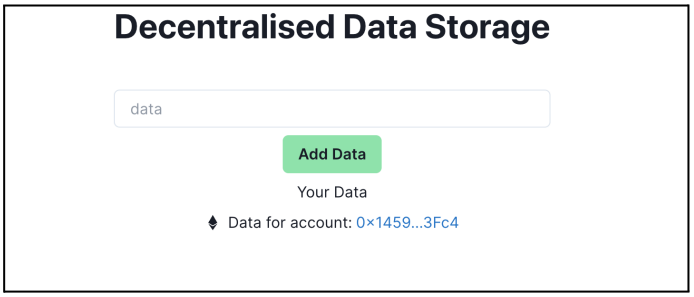
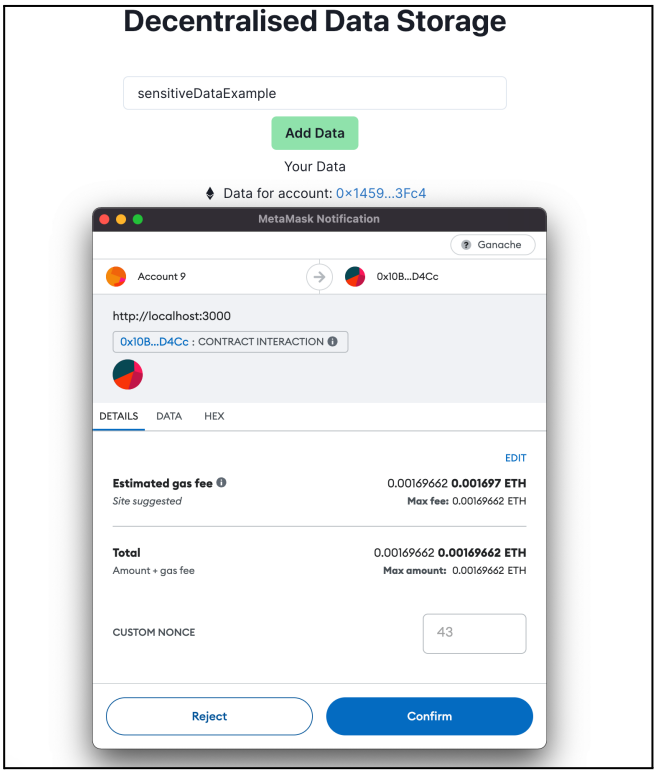
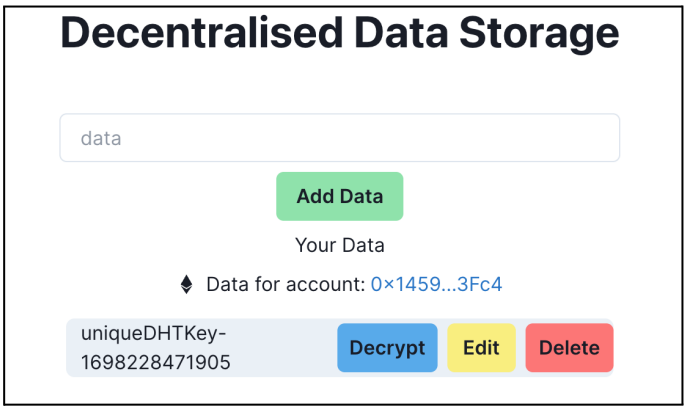
Frontend	**Backend**	**Blockchain**	**Database**	**Cryptography**	**Communication**
NextJs & ChakraUI library	Node Express	Ethereum	IPFS	Mocked on backend server	Libp2p
Web3.js Library		Solidity Smart Contracts	DHTs to be combined with IPFS		

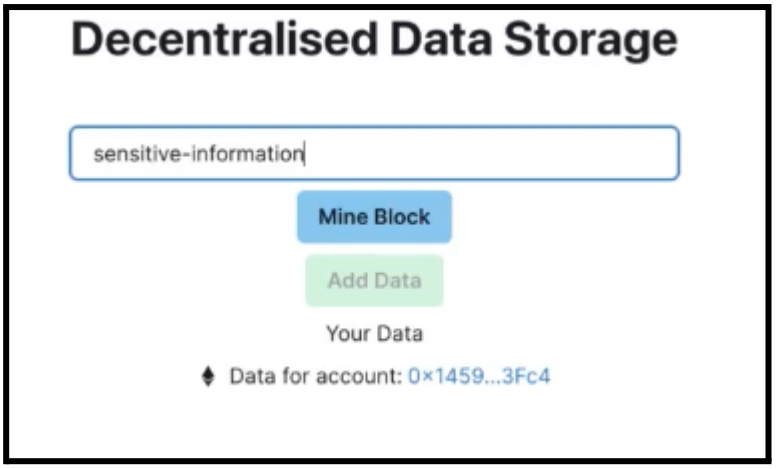
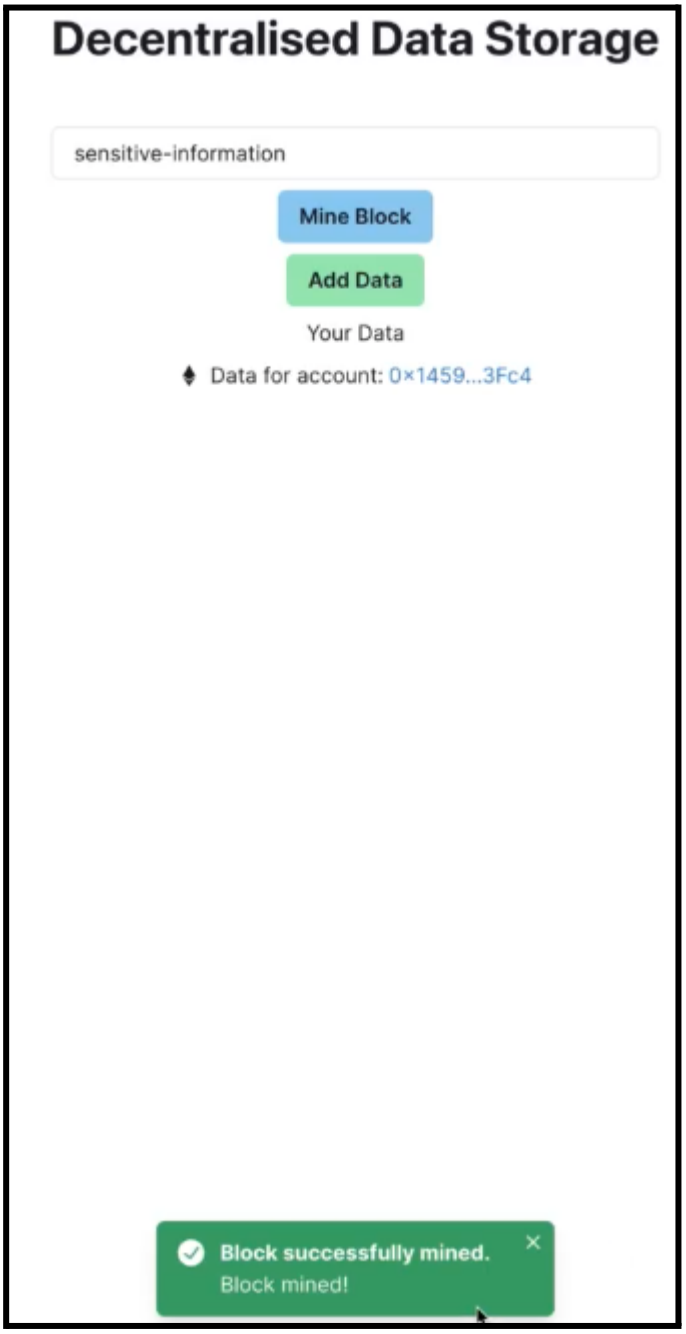
Appendix C: Communication Log

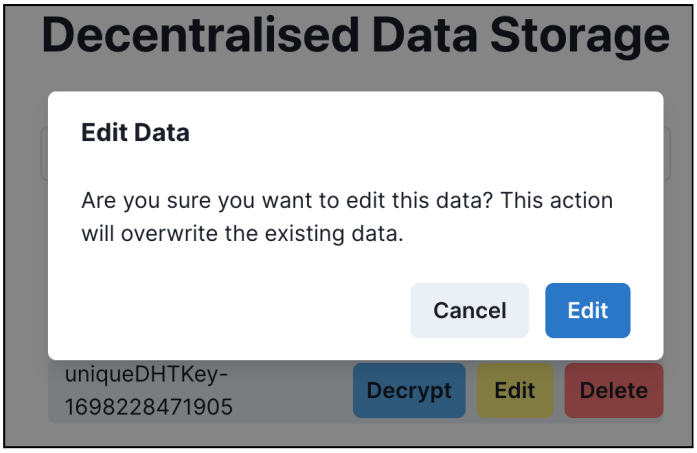
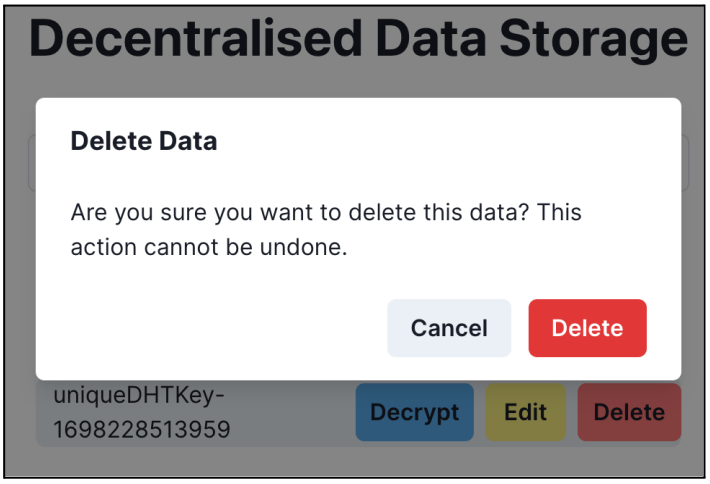
Project Title:	Secure data management and storage using decentralised applications (web3)		
Student Name:	Aryan Gupta	**Supervisor Name:**	Dr Mahira Mohamed Mowjoon
Date	**Event**	**Topic of Communication**	**Outcome**
11/08/2023	Online MS Teams Meeting	Initial development steps	We discussed that my focus for the week would be: setting up the boilerplate application, finalizing the technology stack, construct a high level architecture and to look into the thesis structure
17/08/2023	Online MS Teams Meeting	Review of Technology Stack and Architecture	We reviewed my progress on the initial starter application that I had made via a quick demo. We also reviewed my technology stack and high level project architecture
24/08/2023	Online MS Teams Meeting	Demo of blockchain and backend integration and review of thesis structure	I conducted a demo of the backend and blockchain integration highlighting my smart contract functionality. Additionally we discussed my proposed structure for my thesis
31/08/2023	Online MS Teams Meeting	Live Demo of User Interface Interactions and review of thesis introduction	We went through a demo of the completed user interface and the basic functionality. Discussed progress on thesis' introduction and agreed upon action plan to start more sections
07/09/2023	Online MS Teams Meeting	Thesis progression	Reviewed and discussed thesis progression and formatting.

14/09/2023	Online MS Teams Meeting	Testing Demo	I demoed my progression on my testing functionality for my smart contracts, also received feedback to work on confirmation dialogs and to populate sections of thesis with information that was already available from research and ideation activities
21/09/2023	Online MS Teams Meeting	Review of architecture and new work on thesis sections	We reviewed my new architectural diagrams and thesis iterations. Agreed that focus over STUVAC period would be to split development and thesis work.
05/10/2023	Online MS Teams Meeting	Review of STUVAC Work	Reviewed the work I progressed over STUVAC: application encryption functionality.
19/10/2023	Online MS Teams Meeting	Review on development progress and thesis progress	Discussed how development work is almost complete, just need to complete POW integration. Begun literature / research section of thesis and focus moving forward is to complete it and begin section on findings
26/10/2023	Online MS Teams Meeting	Demo of POW functionality and thesis progression	Demoed functionality of POW as mentioned in previous entry. Showed progress on thesis and discussed next steps to finalise and wrap up.

Appendix D: User Interface

Landing Page	
Web3 Provider Confirmation	
Data Display	

POW	 Decentralised Data Storage sensitive-information Mine Block Add Data Your Data Data for account: 0×1459...3Fc4  Decentralised Data Storage sensitive-information Mine Block Add Data Your Data Data for account: 0×1459...3Fc4 Block successfully mined. Block mined!

Edit Data	
Delete Data	

Appendix E: Development Iteration

Development Iteration Timeline	
System Setup and Configuration	Set up repo Configure dev environment Setup local Ethereum Testnet
Backend Development	Setup Ethereum smart contracts CRUD functions Deploy contracts to local testnet Write unit tests for smart contracts Implement P2P communication protocols Setup nodes for sync
Frontend Development	Design basic UI/UX Create user data input form Implement data operations Connect to Ethereum contracts Display transaction confirmations Display user data in a structured format Implement pagination/infinite scroll Allow users to edit their data entries Implement delete with confirmation
Security Measures	Certificateless cryptography Implement DHTs Ensure consensus protocol security Audit smart contracts
Testing and Quality Assurance	Conduct unit tests Perform integration tests Conduct user acceptance testing Address bugs/issues
Deployment and Launch	Deploy smart contracts to Ethereum mainnet Set up frontend hosting Ensure backend nodes synchronization

Appendix F: Test Results

| Smart Contract Functionality | ```
aryangupta@Anindyas-MBP Capstone % truffle test contracts/test/StorageTest.sol
Using network 'development'.

Compiling your contracts...
==========================
> Everything is up to date, there is nothing to compile.

  StorageTest
    ✔ testCreateData (304ms)
    ✔ testEditData (246ms)
    ✔ testDeleteData (140ms)

  3 passing (9s)
```  |
| Backend Server | ```
aryangupta@Anindyas-MBP backend % yarn test
yarn run v1.22.19
$ NODE_ENV=test jest
 PASS  ./server.test.ts
  Certificateless Crypto Service
    GET /
      ✓ should return a greeting message (26 ms)
    POST /encrypt
      ✓ should encrypt the data and return a DHT key (19 ms)
    POST /decrypt
      ✓ should decrypt the data given a valid DHT key (6 ms)
      ✓ should return a 404 error if the DHT key is invalid (3 ms)
    Mining functionality
      ✓ should return the mining difficulty and previous hash (2 ms)
      ✓ should accept a mining solution (3 ms)

Test Suites: 1 passed, 1 total
Tests:       6 passed, 6 total
Snapshots:   0 total
Time:        2.05 s, estimated 3 s
Ran all test suites.
✨  Done in 4.80s.
```  |

References

Algarni, F., Alharthi, A., Khan, M.A., & Quasim, M.T. (2020). *Decentralised Internet of Things: A Blockchain Perspective.* Springer

Bajpai, V., Doan, T.V., Ott, J., & Psara, Y. (2022). *Towards Decentralised Cloud Storage with IPFS: Opportunities, Challenges, and Future Considerations.* https://arxiv.org/abs/2202.06315

Clarke, I. (1999). *A distributed Decentralised Information Storage and Retrieval System* [Report, University of Edinburgh] https://cs.baylor.edu/~donahoo/classes/5321/papers/C99.pdf

Davies, J., Huffman, T., Leslie, M. (2006). A Comparison Of Replication Strategies for Reliable Decentralised Storage. *Journal of Networks, 1(6)*. http://pages.di.unipi.it/ricci/Replication.pdf

Li, R., Li, H., Song, T., & Sun, L. (2019). *Blockchain for Large-Scale Internet of Things Data Storage and Protection*. https://ieeexplore.ieee.org/document/8404099

LinkChains: Exploring the space of decentralised trustworthy Linked Data. http://oro.open.ac.uk/52928/

OpenJS Foundation. (n.d.). *About Node.js.* https://nodejs.org/en/about

Vercel. (2023). *NextJs Documentation.* https://nextjs.org/docs